

Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

Лабораторная работа №1

по дисциплине «Естественно-языковой интерфейс интеллектуальных
систем»

на тему «Разработка информационно-поисковой системы.
Реализация методов оценки качества её работы
(вариант 22)»

Выполнили студенты гр. 321701

Ковальчук В. Н.
Самович В. М.

Проверил

Крапивин Ю. Б.

Минск 2026

Цель: освоить на практике основные принципы реализации информационно-поисковых систем и методы оценки качества их работы.

Задание (вариант 22): разработать информационно-поисковую систему со следующей комбинацией признаков варианта:

- **Углублённый элемент** – модуль индексирования документов (построение поискового образа документа: нормализация текста, расчёт TF, IDF, весов терминов и вектора документа);
- **Сфера применения** – сеть Интернет (документы поступают через собственного поискового робота, а не вводятся вручную);
- **Стратегия поиска** – векторная модель (TF-IDF и косинусная мера между вектором документа и вектором запроса);
- **Язык** – английский.

Согласно методическим указаниям, система должна:

- а) принимать на входе множество естественно-языковых текстов, по которым осуществляется поиск, и автоматически выделять ключевые слова документов в соответствии с формулой веса термина;
- б) позволять пользователю формулировать запрос на естественном языке и строить по нему поисковый образ запроса (ПОЗ);
- в) выдавать список документов, ранжированных по релевантности запросу в соответствии с векторной моделью поиска, с активной ссылкой на документ и списком слов запроса, найденных в документе;
- г) на основании метрик, применяемых для оценки качества информационно-поисковых систем (методика ROMIP), программно вычислять и отображать в виде таблиц и графиков оценку качества работы системы;
- д) предоставлять простой и доступный пользовательский интерфейс со справочными подсказками.

Структура разработанной системы

Система реализована как набор взаимодействующих по HTTP сервисов, поднимаемых единой командой `docker compose up`. Внутри каждого сервиса выдержано разделение на слои `domain` (чистая логика без I/O), `application` (сценарии использования), `infrastructure` (БД, HTTP-клиенты, обход веб-страниц) и `interface` (маршруты FastAPI). Логика обработки естественного языка (разбиение текста на слова, TF-IDF, косинусная мера, метрики) вынесена в отдельную переиспользуемую библиотеку `packages/nlp_core`, не привязанную ни к одному сервису, а `nlp-service` – лишь тонкая HTTP-обёртка над ней; это и есть углублённый элемент варианта – модуль индексирования документов. Дополнительно к обязательной

для варианта векторной модели на TF-IDF в этом же модуле реализована и вторая модель поиска – на плотных векторных представлениях документа и запроса, которые вычисляются моделью Qwen3-Embedding-8B; она предназначена для сравнения качества обоих подходов на одной и той же тестовой коллекции и представляет собой дополнительное углубление этого элемента сверх минимальных требований варианта.

Структура системы на уровне контейнеров (C4, уровень 2) показана на рисунке 1.

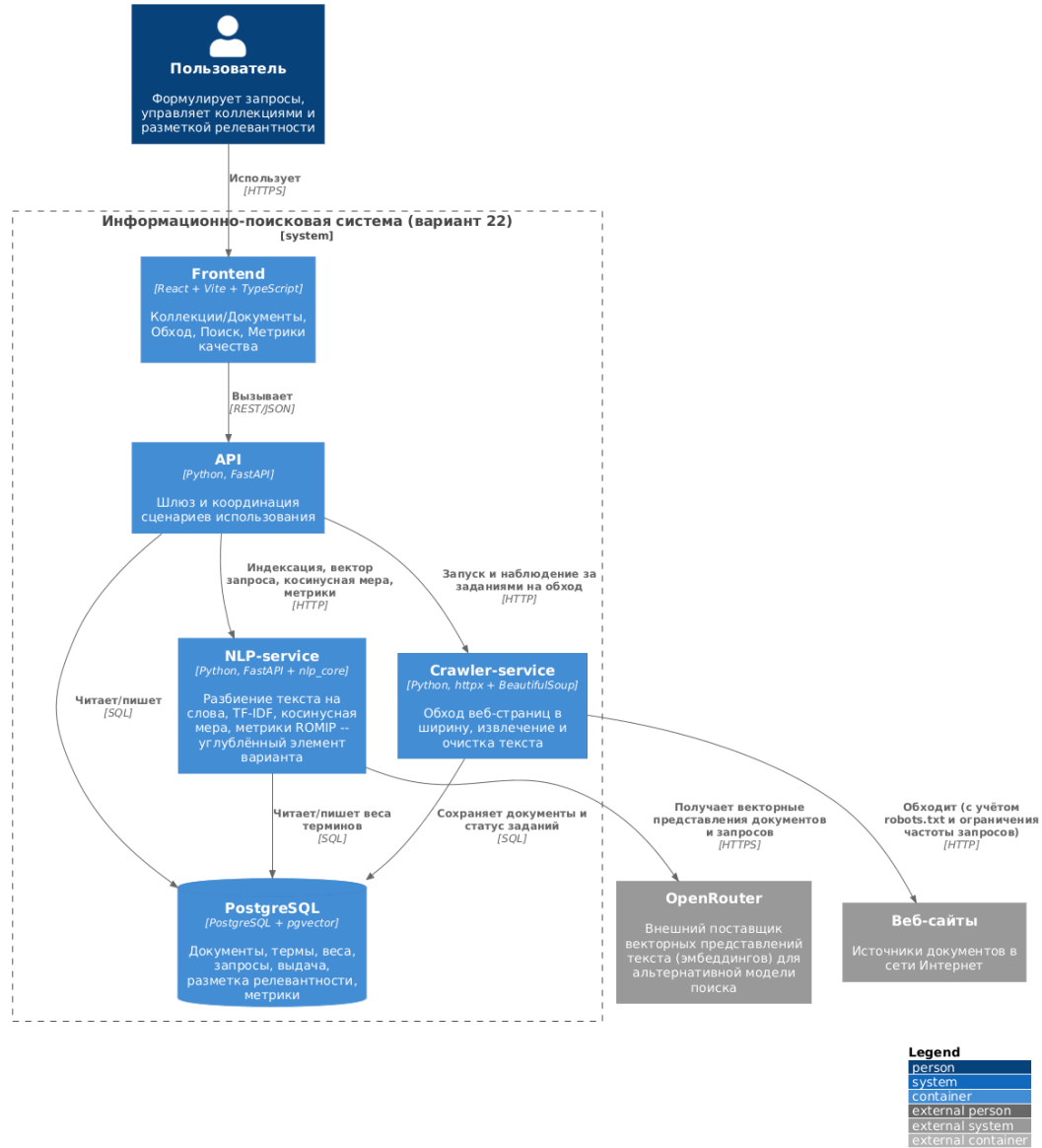


Рисунок 1 – Диаграмма контейнеров разработанной системы (нотация C4)

Структура базы данных

Модель данных спроектирована как мультиколлекционная – не «документы для поиска», а «документы» вообще. Схема ключевых таблиц и связей между ними показана на рисунке 2 в нотации Crow's Foot.

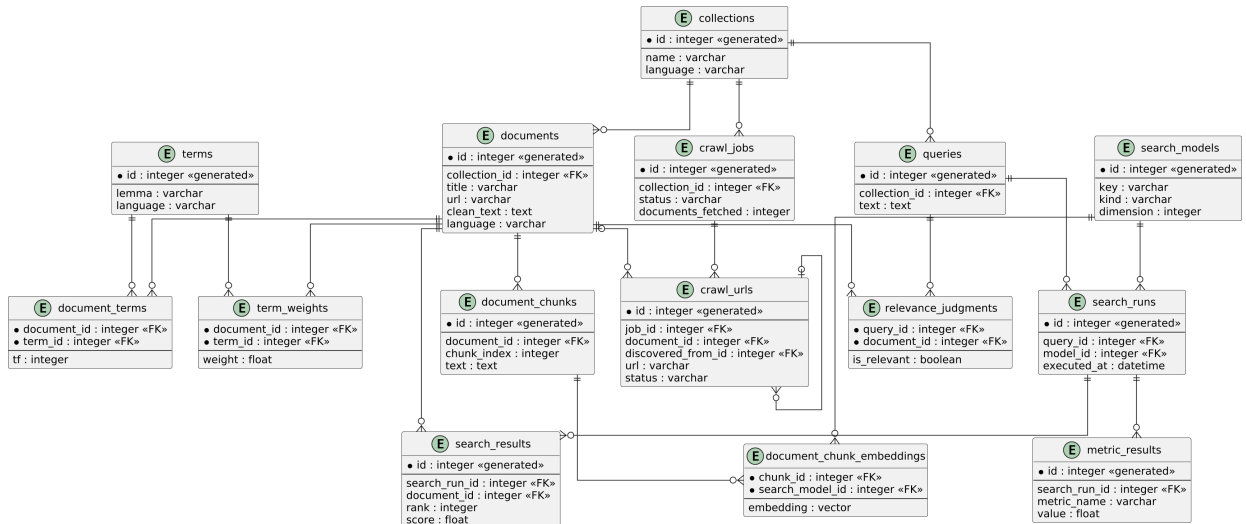


Рисунок 2 – ER-диаграмма базы данных (нотация Crow's Foot)

Пара `document_terms.tf + term_weights.weight` – это то, что в методических указаниях называется поисковым образом документа (вектором документа); таблица `terms` с агрегатом `COUNT(DISTINCT document_id)` – источник инверсной частоты термина (формула 1.5). Таблицы `search_models` и `document_chunk_embeddings` хранят плотные векторные представления для второй, эмбединговой модели поиска (Qwen3-Embedding-8B), описанной выше; `crawl_jobs/crawl_urls` – параметры и живой прогресс выполнения задания на обход и очередь обхода (состояние каждого URL: `queued/fetching/success/failed`).

Основные алгоритмы

Обход веб-страниц. Обход выполняется по алгоритму поиска в ширину (BFS) с ограничением по глубине и количеству документов. Стартовые URL кладутся в `crawl_urls` с `depth=0`; робот извлекает из очереди URL с наименьшей глубиной, проверяет `robots.txt` и ограничивает частоту запросов к одному домену, скачивает и очищает текст, сохраняет документ и кладёт исходящие ссылки в очередь с `depth+1`, если новая глубина не превышает `max_depth` и общий счётчик документов не превысил `max_documents`. Повторы исключаются по нормализованному URL в рамках одного задания на обход.

Предварительная обработка текста. Текст документа проходит очистку HTML, разбиение на предложения и слова, приведение к нижнему регистру, удаление стоп-слов и лемматизацию (spaCy, модель `en_core_web_sm` для английского языка согласно варианту).

Индексация. Для каждого термина i рассчитывается инверсная частота

$$B_i = \log\left(\frac{N}{P_i}\right), \quad (1)$$

где N – число документов в коллекции, P_i – число документов, содержащих термин i ; вес термина в документе j –

$$A_i^j = Q_i^j \cdot B_i, \quad (2)$$

где Q_i^j – частота термина i в документе j (TF). Итоговый вектор документа D – вектор весов A_i^j , нормированный на собственную евклидову норму; в обозначениях методических указаний (где термин k , документ d , $N_{dk} \equiv Q_i^j$, $N_k \equiv P_i$) это записывается как

$$w_{dk} = \frac{N_{dk} \log(N/N_k)}{\sqrt{\sum_j (N_{dj} \log(N/N_j))^2}}, \quad (3)$$

где сумма в знаменателе – та же евклидова норма вектора весов документа d , то есть числитель – это A_i^j из формулы (2), а всё выражение – нормированный A_i^j . Такая нормализация впоследствии позволяет свести косинусную меру к скалярному произведению нормированных векторов. Реализация этих формул – чистые функции без побочных эффектов, покрытые модульными тестами.

Поиск. Запросу пользователя ставится в соответствие бинарный вектор $Q = \{w_{q1}, \dots, w_{qn}\}$, где $w_{qj} = 1$, если j -е слово присутствует в запросе, и $w_{qj} = 0$ в противном случае – то есть вектор запроса не взвешивается по IDF, в отличие от вектора документа. Мера схожести документа D и запроса Q вычисляется как косинус угла между векторами:

$$r(D, Q) = \frac{(D, Q)}{\|D\| \cdot \|Q\|}, \quad (4)$$

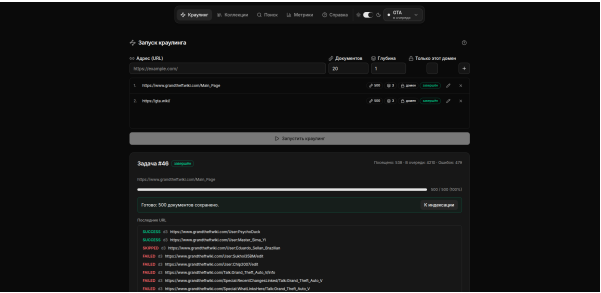
где (D, Q) – скалярное произведение векторов, $\|D\|$ и $\|Q\|$ – их евклидовы нормы. Документы сортируются по убыванию $r(D, Q)$; в выдачу попадают топ- K документов с фрагментом текста (первые ≈ 300 символов), в котором подсвечены найденные слова запроса.

Оценка качества. Метрики реализованы как чистые функции от (ранжированный список, множество релевантных документов): Precision,

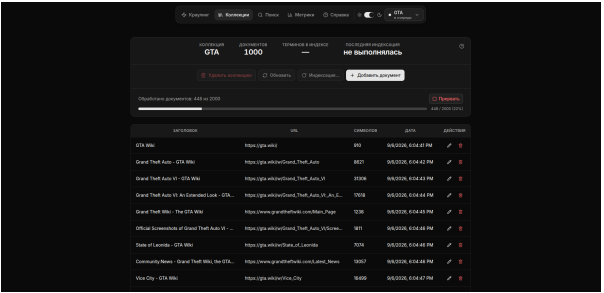
Recall, F1 на топ- K (отдельно $K = 5$ и $K = 10$), Average Precision (AP), R-Precision и 11-точечная интерполированная кривая Precision/Recall (модуль `nlp_core.metrics`), в соответствии с методикой ROMIP.

Тестовая коллекция документов

Для оценки качества собрана коллекция **GTA**: 1000 документов на английском языке, 18 071 уникальная лемма в словаре, средний объём документа – $\approx 11\,306$ символов очищенного текста. Коллекция наполнена двумя независимыми запусками обхода, каждый ограничен своим доменом (рисунок 3): с `grandtheftwiki.com` (глубина 3, лимит 500 документов; получено 500 из 538 посещённых URL, 479 ошибок – в основном служебные страницы редактирования и обсуждения) и с `gta.wiki` (глубина 3, лимит 500 документов; получено 500 из 539 посещённых URL, ошибок не было). Итоговая коллекция и её документы также показаны на том же рисунке.



(а) Запуск обхода



(б) Наполненная коллекция

Рисунок 3 – Сбор тестовой коллекции **GTA** в интерфейсе системы

Для коллекции сформулировано 5 тестовых запросов на английском языке: *GTA series publisher*, *GTA 5 first day sales volume*, *GTA VI release date*, *Vice City highway system*, *Megamundo building*. Для каждого запроса выполнен поиск обеими реализованными моделями (TF-IDF и Qwen3-Embedding-8B), и в режиме разметки на странице «Поиск» отмечены документы, признанные релевантными (рисунок 4) – итого 32 отметки (12, 3, 8, 4 и 5 документов по перечисленным запросам соответственно), хранящиеся в таблице `relevance_judgments`. Интерфейс поддерживает только положительную отметку «релевантно»; все остальные документы из выдачи, не отмеченные пользователем, считаются нерелевантными при расчёте метрик.

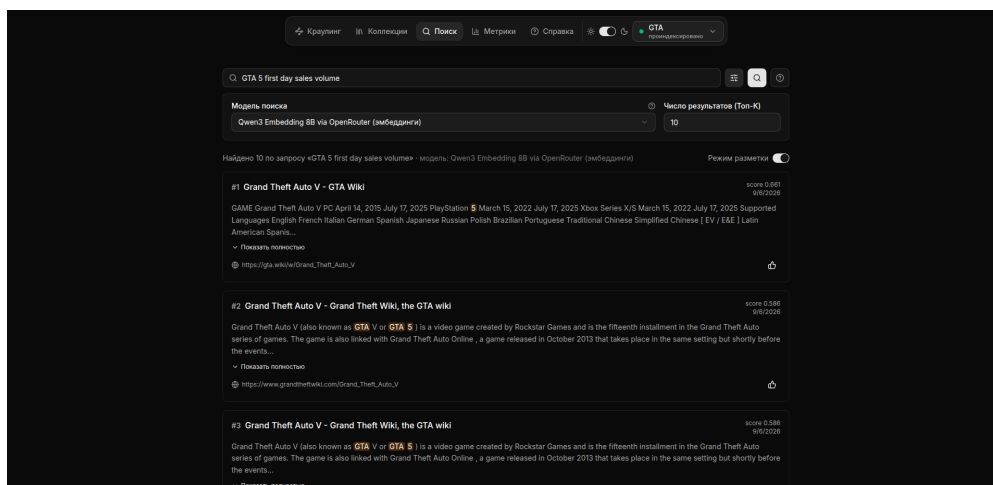


Рисунок 4 – Поиск по коллекции с выбором модели и разметкой релевантности

Результаты оценки по метрикам

Метрики посчитаны для обеих реализованных моделей поиска – обязательной (TF-IDF + косинусная мера) и дополнительной (Qwen3-Embedding-8B через OpenRouter) – на одной и той же тестовой коллекции **GTA** и одном и том же наборе из 5 размеченных запросов; ни один запрос не пришлось исключать из агрегатов – у каждого нашёлся хотя бы один документ, отмеченный релевантным. Средние по запросам значения приведены в таблице 1; тот же расчёт в интерфейсе системы – на рисунке 5.

Таблица 1 – Средние метрики качества по 5 тестовым запросам коллекции **GTA**

Метрика	TF-IDF + косинусная мера	Qwen3-Embedding-8B
MAP	0,442	0,842
R-Precision	0,475	0,775
P@5	0,520	0,760
P@10	0,300	0,580
R@5	0,450	0,658
R@10	0,492	0,950
F1@5	0,465	0,666
F1@10	0,358	0,681

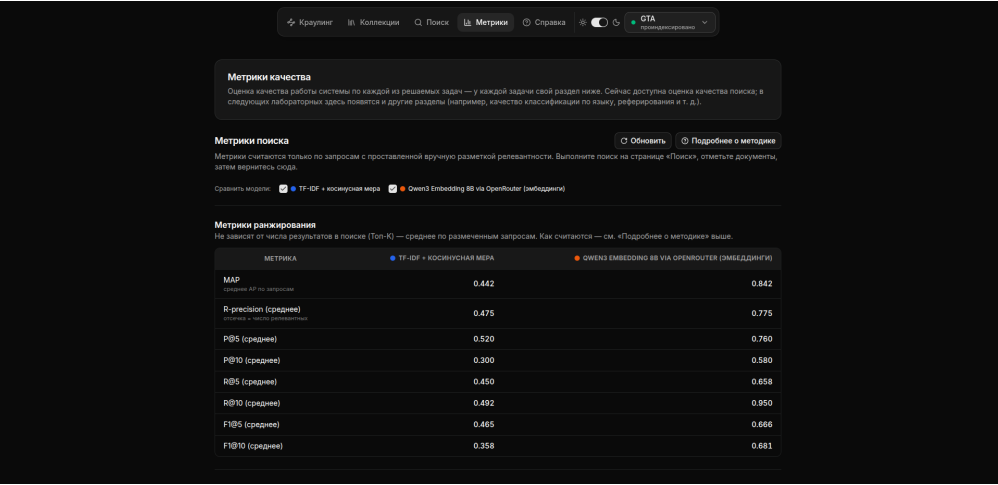
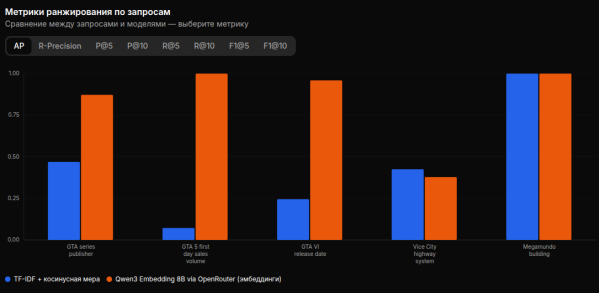
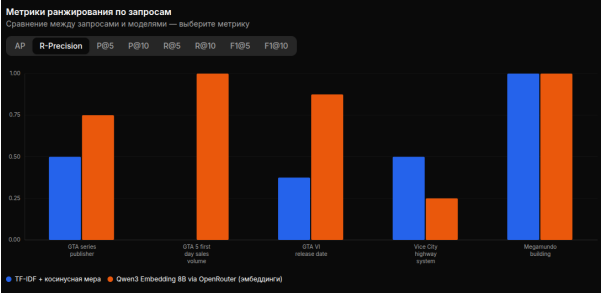


Рисунок 5 – Таблица метрик качества поиска в интерфейсе системы

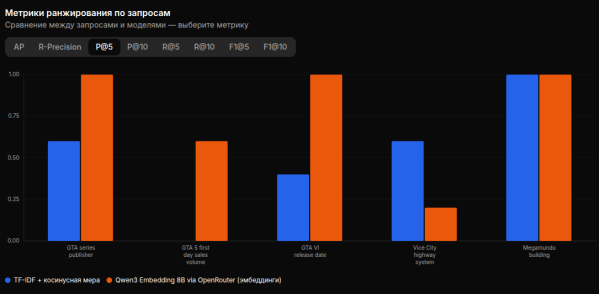
Графическое представление метрик по каждому из 5 запросов – на рисунках 6 и 7 (синий – TF-IDF, оранжевый – Qwen3-Embedding-8B).



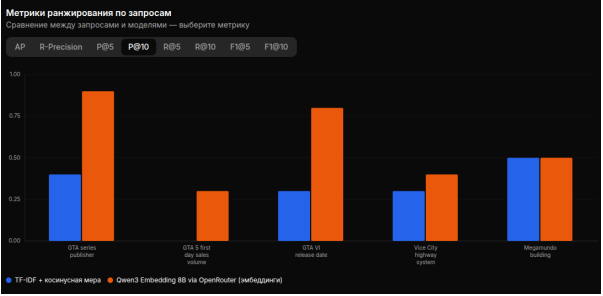
(a) AP



(б) R-Precision

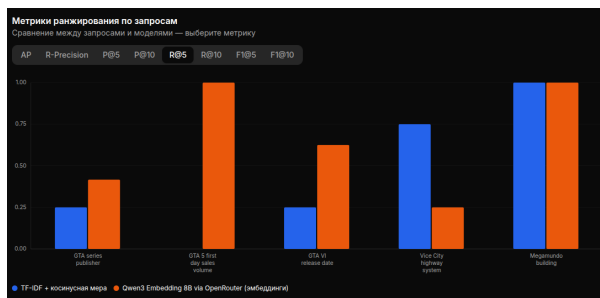


(в) P@5

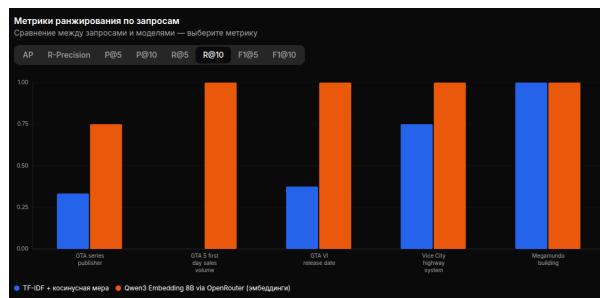


(г) P@10

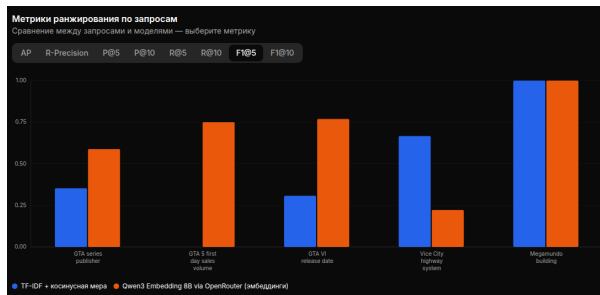
Рисунок 6 – Метрики ранжирования по запросам: AP, R-Precision, P@5, P@10



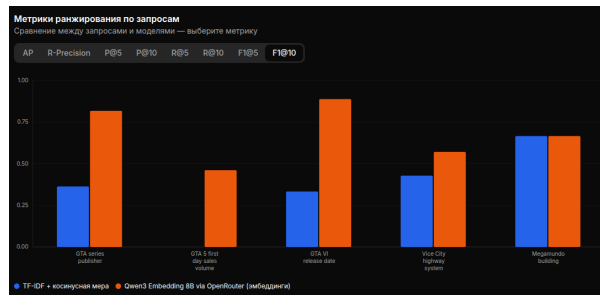
(a) R@5



(б) R@10



(в) F1@5



(г) F1@10

Рисунок 7 – Метрики ранжирования по запросам: R@5, R@10, F1@5, F1@10

11-точечная интерполированная кривая Precision/Recall, усреднённая по размеченным запросам, показана на рисунке 8: кривая Qwen3-Embedding-8B лежит выше кривой TF-IDF практически на всём диапазоне полноты, кроме самого высокого уровня, где обе модели сближаются.

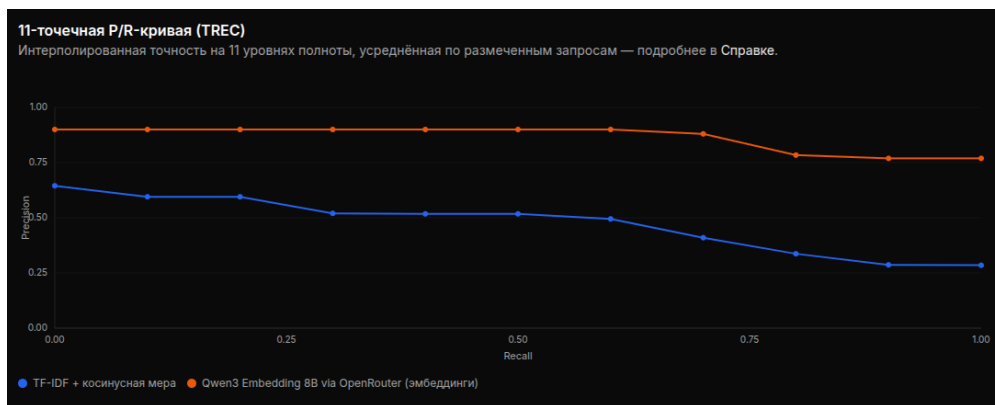


Рисунок 8 – 11-точечная интерполированная кривая Precision/Recall

Анализ результатов и предложения по улучшению

а) **Эмбеддинговая модель в среднем точнее обязательной.** Qwen3-Embedding-8B превосходит TF-IDF по всем восьми агрегированным метрикам (таблица 1): MAP 0,842 против 0,442, R-Precision 0,775 против

0,475 и так далее. Ожидаемый результат: векторные представления учитывают смысл запроса, а не только буквальное совпадение слов с текстом документа.

б) **Главный источник разрыва – рассинхрон словаря (vocabulary mismatch).** На запросе *"GTA 5 first day sales volume"* TF-IDF находит единственный релевантный документ только на дальних позициях выдачи ($P@5 = P@10 = 0$, $AP = 0,07$), тогда как Qwen – уже в топ-5 ($AP = 1,00$, рисунок 6). Страница, видимо, описывает продажи другими словами (например, суммой выручки вместо *"sales volume"*), не пересекающимися буквально с текстом запроса, – TF-IDF в принципе не может найти документ без общих термов, а эмбединги устойчивы к такой перефразировке.

в) **Но не всегда эмбединги выигрывают.** На запросе *"Vice City highway system"* TF-IDF обходит Qwen по всем метрикам ($P@5$ 0,60 против 0,20, AP 0,42 против 0,38, R -Precision 0,50 против 0,25) – для узких формулировок, дословно совпадающих с текстом страницы, точное совпадение термов оказывается надёжнее семантического сходства. Предложение: реализовать гибридное ранжирование (например, взвешенную сумму или reciprocal rank fusion косинусных оценок обеих моделей), чтобы не терять точность на подобных запросах.

г) **Что можно добавить в систему для её улучшения.**

- расширение запроса синонимами и близкими по смыслу словами (например, через векторы слов spaCy или WordNet) – частично закрыло бы разрыв словаря из п. б без отказа от быстрой и прозрачной модели на TF-IDF;

- переход от бинарной релевантности к градуированной шкале и метрике nDCG, чтобы различать «точное совпадение» и «частично по теме», а не только «релевантно / нерелевантно»;

- приближённый индекс поиска по векторным представлениям (ivfflat/hnsw в pgvector) – сейчас поиск по эмбедингам выполняется полным перебором, что приемлемо только для учебного масштаба коллекции;

- полуавтоматическое пополнение эталонной разметки – объединение (pooling) топ- K выдачи обеих моделей в единый список кандидатов на разметку, чтобы быстрее набрать рекомендованные методикой 15–20 тестовых запросов, не размечая вручную каждый документ с нуля;

- многопользовательский режим с сохранением истории запросов и персональной разметкой, если систему развивать за пределы учебного проекта.

д) **Где систему можно применить.**

- внутрикорпоративный поиск по базе знаний или документации – коллекция и краулинг уже спроектированы так, чтобы наполняться из произвольного набора стартовых страниц одного источника (как коллекция

GTA наполнена с `gta.wiki` и `grandtheftwiki.com`);

- поиск по тематическим wiki-сайтам и базам знаний, где важна устойчивость к перефразировкам запроса пользователя – здесь особенно уместна эмбединговая модель (п. а);

- управляемый сбор и полнотекстовый поиск по узкой предметной области сети Интернет (мониторинг новостей по теме, дайджесты) – за счёт ограничения краулинга по домену и глубине обхода;

- учебный полигон для последующих лабораторных работ курса (определение языка, реферирование, машинный перевод, синтез и распознавание речи) – архитектура и модель данных уже мультиколлекционные и переиспользуемые, без необходимости поднимать отдельную инфраструктуру под каждую лабораторную.

Использованные готовые компоненты

Таблица 2 – Библиотеки и программные средства, использованные при реализации

Компонент	Назначение
FastAPI	Асинхронная библиотека для построения API, автоматическая генерация OpenAPI-схемы для клиентского приложения.
spaCy (<code>en_core_web_sm</code>)	Разбиение текста на слова, лемматизация, удаление стоп-слов, определение частей речи для английского языка.
PostgreSQL + pgvector	Реляционное хранилище термов, весов и документов; расширение <code>pgvector</code> хранит плотные векторные представления документов и запросов для второй модели поиска (Qwen3-Embedding-8B), реализованной для сравнения с обязательной моделью на TF-IDF.
Alembic	Версионирование схемы базы данных.
httpx + BeautifulSoup	HTTP-клиент и разбор HTML для поискового робота.
React + Vite + TypeScript	Клиентское веб-приложение.
pytest	Модульное тестирование доменного слоя (<code>nlp_core</code>) без поднятия инфраструктуры.
Docker Compose	Единая среда запуска всех сервисов и базы данных.

Требование методических указаний о доступном интерфейсе со справочными подсказками реализовано отдельной страницей «Справка» (рисунок 9): она описывает типичный порядок действий и назначение каждой страницы приложения.

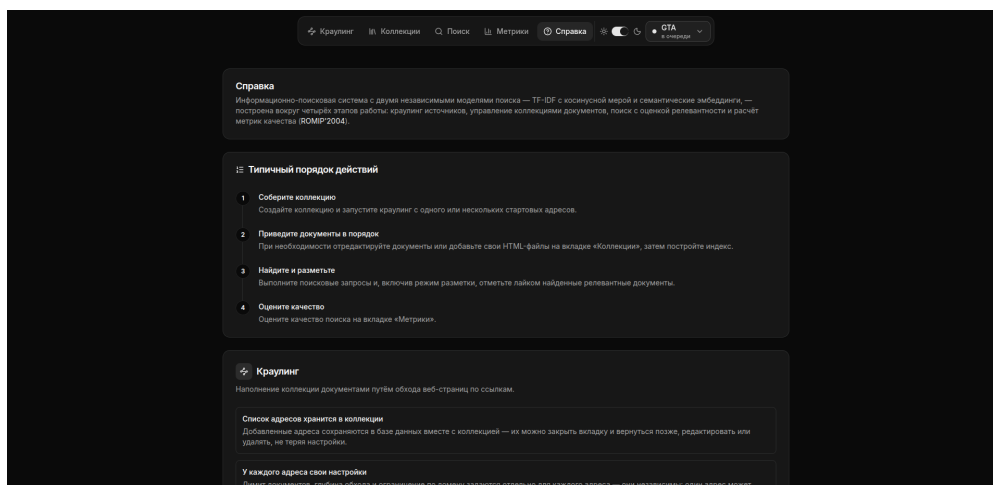


Рисунок 9 – Страница «Справка» интерфейса системы

Выводы

В ходе выполнения лабораторной работы разработана информационно-поисковая система по варианту 22: собственный поисковый робот наполняет коллекцию англоязычными документами, модуль индексирования строит поисковый образ каждого документа по формулам инверсной частоты термина и веса термина (1.5–1.6), а поиск по естественно-языковому запросу реализован через векторную модель – построение поискового образа запроса и ранжирование документов по косинусной мере. Дополнительно к обязательной модели на TF-IDF реализована вторая модель поиска – на плотных векторных представлениях, вычисляемых моделью Qwen3-Embedding-8B, – для последующего сравнения качества обоих подходов. Все ключевые формулы вынесены в переиспользуемую библиотеку в виде чистых, покрытых тестами функций, что позволило проверить их корректность независимо от инфраструктуры (БД, HTTP).

Реализована и опробована на собранной коллекции GTA (1000 документов) оценка качества по методике ROMIP: Precision/Recall/F1 на топ- K , Average Precision, R-Precision и интерполированная кривая Precision/Recall. По 5 тестовым запросам дополнительная модель на плотных векторных представлениях (Qwen3-Embedding-8B) превзошла обязательную модель на TF-IDF по всем агрегированным метрикам (MAP 0,842 против 0,442, R-Precision 0,775 против 0,475) – в основном за счёт устойчивости к перефразировкам запроса, которые TF-IDF не находит из-за отсутствия общих слов с текстом документа. При этом на одном из пяти запросов, дословно совпадающем с формулировками в тексте документов, точнее оказался именно TF-IDF – то есть ни одна из моделей не является безусловно лучшей,

и разумным следующим шагом является гибридное ранжирование, комбинирующее обе оценки. Также подтвердилось, что более раннее исправление повторного использования одинаковых по тексту запросов действительно снимает вырождение Recall в 1 на достаточно большой коллекции. Набор тестовых запросов (5) при этом всё ещё меньше рекомендованных методикой 15–20, что ограничивает статистическую надёжность найденных различий.